

COLD STORAGE COMMODITIES AND CHAMBER MANAGEMENT SYSTEM

A COURSE PROJECT REPORT

Submitted in partial fulfillment of the requirements
for the course

DATABASE MANAGEMENT SYSTEMS

Submitted by	: Manoj Sri Satya Charab
Roll No.	: 25WU0102169
Section	: AIML PANTHERS
Course	: DBMS
Programme	: B.Tech CSE (AIML)
Academic Year	: 2025 – 2029

Woxsen University

Hyderabad, India

DECLARATION

I hereby declare that the project titled **“Cold Storage Commodities and Chamber Management System”** is an individual academic project developed for the Database Management Systems course. The database design, entity–relationship model, SQL implementation, sample data, and documented outputs presented in this report represent the project work supplied for this submission.

Student	Roll No.	Section	Course
Manoj Sri Satya Charab	25WU0102169	AIML Panthers	DBMS

ABSTRACT

The Cold Storage Commodities and Chamber Management System is a relational database application designed to organize and manage the storage of commodities in temperature-controlled facilities. The system maintains customers, commodities, lots, chambers, physical storage locations, allocations, temperature observations, inspections, releases, rates, invoices, and payments. It uses primary keys and foreign keys to preserve relationships and integrity between operational records.

The project is implemented using MySQL. Business information is represented through normalized relational tables, while the ER diagram models the relationships and cardinalities among the major entities. The database also uses constraints such as NOT NULL, UNIQUE, CHECK, DEFAULT, and FOREIGN KEY constraints to reduce invalid or inconsistent data. Sample data and query outputs demonstrate the working state of the database.

1. INTRODUCTION

Cold-storage facilities handle commodities that must be maintained within suitable temperature ranges. Manual or fragmented record keeping can make it difficult to track where a lot is stored, which customer owns it, the condition of the commodity, temperature history, release activity, billing, and payment status.

This project models those activities as an integrated relational database. A customer owns lots; each lot identifies a commodity and quantity; allocations connect lots to chambers and locations; temperature logs and inspections record storage conditions; releases initiate billing; invoices record amounts due; and payments record settlement activity.

The database is intended to provide a consistent source of information for an administrator managing the cold-storage operation.

2. PROBLEM STATEMENT

A cold-storage operation needs to maintain accurate records of customers, commodities, lots, storage chambers, exact locations, environmental conditions, inspections, releases, billing, and payments. If these records are maintained separately, duplicate data, missing relationships, and difficulty in tracking a lot through its lifecycle can occur. The proposed database centralizes these records and connects them through relational keys.

3. OBJECTIVES

No.	Objective
1	Maintain customer and commodity master records.
2	Track incoming lots with quantity, arrival date, ownership, and status.
3	Represent chambers and physical rack/location capacity.
4	Allocate lots to suitable chambers and locations.
5	Record temperature observations against allocations.
6	Record inspections and quality/condition remarks.
7	Track commodity release transactions.
8	Maintain storage rates and effective dates.
9	Generate and track invoices for releases.
10	Record payment details and payment status.

4. SCOPE

Included	Not explicitly implemented in supplied SQL
Customer management	User authentication/role management
Commodity master data	Online payment gateway integration
Lot and ownership tracking	IoT sensor integration
Chamber and location management	Automatic alert/notification service
Allocation tracking	Mobile application
Temperature and inspection records	Predictive analytics
Release, invoice and payment records	External ERP integration
Rate management	

5. SYSTEM USERS AND FUNCTIONAL REQUIREMENTS

User	Access / responsibility
Administrator	Primary system operator; manages master data and operational records across the database.
Customer	Represents the owner of stored commodities and associated lots; customer details are maintained in the system.

5.1 Functional requirements

ID	Requirement
FR-01	Store and retrieve customer information.
FR-02	Store commodity categories, units, and permitted temperature ranges.
FR-03	Create lots and associate each lot with a customer and commodity.
FR-04	Maintain chamber capacity, chamber type, and operating temperature range.
FR-05	Maintain rack/location information within chambers.
FR-06	Allocate lots to chambers and locations with start/end dates and status.
FR-07	Record temperature observations for allocations.
FR-08	Record inspections and remarks for allocations.
FR-09	Record releases and released quantities.
FR-10	Maintain commodity/chamber-type storage rates.
FR-11	Create invoices linked to releases.
FR-12	Record one or more payments against an invoice.

5.2 Non-functional requirements

Area	Requirement
Integrity	Primary/foreign keys and CHECK/UNIQUE/NOT NULL constraints should preserve valid data.
Consistency	Related records should be connected using foreign-key relationships.
Usability	Data should be represented in clear relational tables that can be queried through MySQL Workbench.
Maintainability	The schema should separate master data, operational records, monitoring records, and billing records.
Scalability	The relational design can be extended with additional customers, commodities, lots, chambers, and transactions.

6. TECHNOLOGY STACK

Component	Technology
Database Management System	MySQL
Database client / IDE	MySQL Workbench
Database	ColdStorageDB
Data model	Relational model
Design model	Entity–Relationship (ER) model
Language	SQL

7. ENTITY–RELATIONSHIP DESIGN

The ER model represents the principal entities, their attributes, and relationships. Underlined attributes in the supplied diagram denote primary keys; dashed/underlined attributes denote foreign-key relationships as indicated in the diagram. The major operational flow is Customer → Lot → Allocation → Temperature/Inspection → Release → Invoice → Payment, with Commodity, Chamber, Location, and Rate supporting storage operations.

Cold storage commodity and chamber management system
 Underlined = primary key Dashed = foreign key 1 / M = cardinality

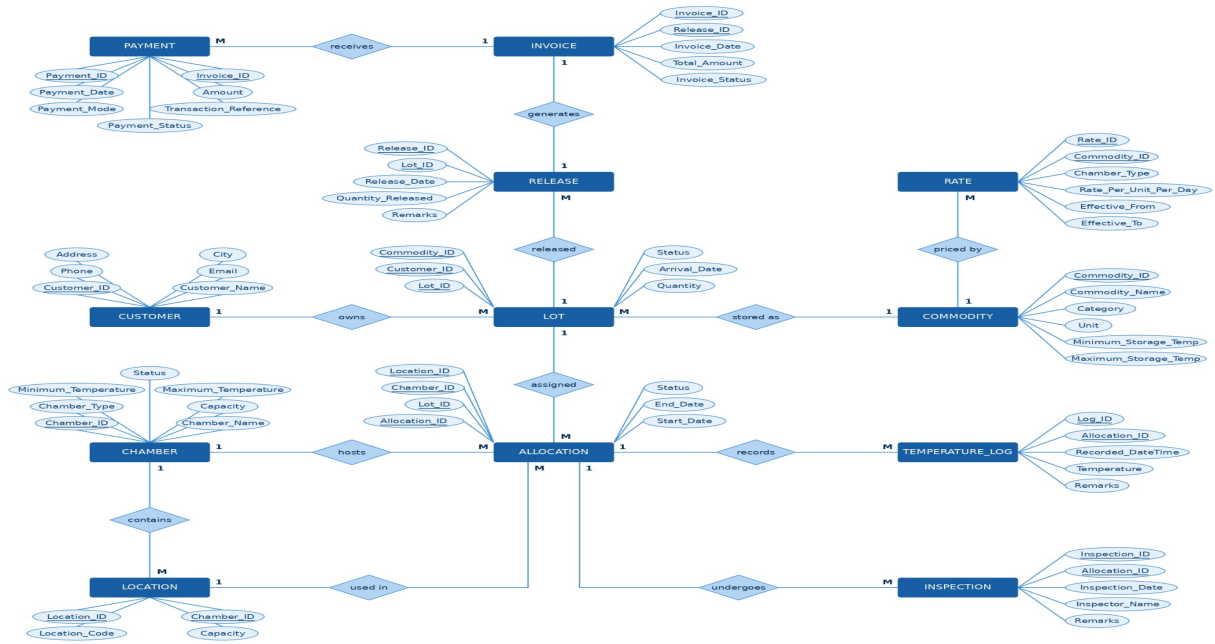


Figure 1. ER diagram of the Cold Storage Commodities and Chamber Management System.

7.1 Major relationships

Relationship	Meaning
Customer — owns — Lot	A customer can own multiple lots; each lot references a customer.
Commodity — stored as — Lot	A lot identifies the commodity being stored.
Lot — assigned — Allocation	An allocation records where and when a lot is stored.
Chamber — hosts — Allocation	A chamber can host multiple allocations.
Location — used in — Allocation	An allocation identifies a physical storage location.
Allocation — records — Temperature_Log	An allocation can have multiple temperature observations.
Allocation — undergoes — Inspection	An allocation can have inspection records.
Lot — released — Release	A release records quantity removed from a lot.
Release — generates — Invoice	An invoice is generated for a release.
Invoice — receives — Payment	An invoice can have payment records.
Commodity — priced by — Rate	Rates are maintained by commodity and chamber type.

8. RELATIONAL DATABASE DESIGN

The following tables represent the supplied implementation. The report uses tabular descriptions rather than repeating every row of the sample data.

8.1 CUSTOMER

Attribute	Data Type	Key / Constraint	Nullable
Customer_ID	INT	PK	No
Customer_Name	VARCHAR(100)	—	No
Phone	VARCHAR(15)	—	No
Email	VARCHAR(100)	—	Yes
Address	VARCHAR(200)	—	Yes
City	VARCHAR(50)	—	No

8.2 COMMODITY

Attribute	Data Type	Key / Constraint	Nullable
Commodity_ID	INT	PK	No
Commodity_Name	VARCHAR(50)	UNIQUE	No
Category	VARCHAR(50)	—	No
Unit	VARCHAR(20)	—	No
Minimum_Storage_Temperature	DECIMAL(5,2)	—	No
Maximum_Storage_Temperature	DECIMAL(5,2)	—	No

8.3 CHAMBER

Attribute	Data Type	Key / Constraint	Nullable
Chamber_ID	INT	PK	No
Chamber_Name	VARCHAR(50)	UNIQUE	No
Chamber_Type	VARCHAR(50)	—	No
Capacity	INT	—	No
Minimum_Temperature	DECIMAL(5,2)	—	No

Attribute	Data Type	Key / Constraint	Nullable
Maximum_Temperature	DECIMAL(5,2)	—	No
Status	VARCHAR(20)	DEFAULT	No

8.4 RATE

Attribute	Data Type	Key / Constraint	Nullable
Rate_ID	INT	PK	No
Commodity_ID	INT	FK → Commodity	No
Chamber_Type	VARCHAR(50)	—	No
Rate_Per_Unit_Per_Day	DECIMAL(10,2)	—	No
Effective_From	DATE	—	No
Effective_To	DATE	—	Yes

8.5 LOT

Attribute	Data Type	Key / Constraint	Nullable
Lot_ID	VARCHAR(10)	PK	No
Customer_ID	INT	FK → Customer	No
Commodity_ID	INT	FK → Commodity	No
Quantity	DECIMAL(10,2)	—	No
Arrival_Date	DATE	—	No
Status	VARCHAR(20)	DEFAULT	No

8.6 LOCATION

Attribute	Data Type	Key / Constraint	Nullable
Location_ID	INT	PK	No
Chamber_ID	INT	FK → Chamber	No
Location_Code	VARCHAR(20)	UNIQUE with Chamber_ID	No
Capacity	INT	—	No

8.7 ALLOCATION

Attribute	Data Type	Key / Constraint	Nullable
Allocation_ID	INT	PK	No
Lot_ID	VARCHAR(10)	FK → Lot	No
Chamber_ID	INT	FK → Chamber	No
Location_ID	INT	FK → Location	No
Start_Date	DATE	—	No
End_Date	DATE	—	Yes
Status	VARCHAR(20)	DEFAULT	No

8.8 TEMPERATURE_LOG

Attribute	Data Type	Key / Constraint	Nullable
Log_ID	INT	PK	No

Attribute	Data Type	Key / Constraint	Nullable
Allocation_ID	INT	FK → Allocation	No
Recorded_DateTime	DATETIME	—	No
Temperature	DECIMAL(5,2)	—	No
Remarks	VARCHAR(200)	—	Yes

8.9 INSPECTION

Attribute	Data Type	Key / Constraint	Nullable
Inspection_ID	INT	PK	No
Allocation_ID	INT	FK → Allocation	No
Inspection_Date	DATE	—	No
Inspector_Name	VARCHAR(100)	—	No
Condition	VARCHAR(50)	—	No
Remarks	VARCHAR(200)	—	Yes

8.10 RELEASE

Attribute	Data Type	Key / Constraint	Nullable
Release_ID	INT	PK	No
Lot_ID	VARCHAR(10)	FK → Lot	No
Release_Date	DATE	—	No
Quantity_Released	DECIMAL(10,2)	—	No
Remarks	VARCHAR(200)	—	Yes

8.11 INVOICE

Attribute	Data Type	Key / Constraint	Nullable
Invoice_ID	INT	PK	No
Release_ID	INT	FK → Release; UNIQUE	No
Invoice_Date	DATE	—	No
Total_Amount	DECIMAL(12,2)	—	No
Invoice_Status	VARCHAR(20)	DEFAULT	No

8.12 PAYMENT

Attribute	Data Type	Key / Constraint	Nullable
Payment_ID	INT	PK	No
Invoice_ID	INT	FK → Invoice	No
Payment_Date	DATE	—	No
Amount	DECIMAL(12,2)	—	No
Payment_Mode	VARCHAR(30)	—	No
Transaction_Reference	VARCHAR(50)	—	Yes
Payment_Status	VARCHAR(20)	DEFAULT	No

Implementation note — Receipt table

Important: the requested table list includes a *Receipt* table, but the SQL supplied for this report does not contain a CREATE TABLE Receipt statement, INSERT statements for Receipt, or a Receipt entity in the supplied ER diagram. The implemented database therefore contains 12 tables, not 13. This report does not falsely claim Receipt is implemented. If the faculty rubric requires Receipt as a physical table, it should be added to the SQL and ER model before final submission.

9. SAMPLE DATA

The supplied screenshots demonstrate populated records in the implemented database. The following tables summarize the data supplied by the project SQL.

CUSTOMER

ID	Name	Phone	Email	City
1	Kohli	9876543210	kohli@email.com	Nagpur
2	Pawan	9123456780	pawan@email.com	Pune
3	Vijay	9988776655	vijay@email.com	Hyderabad
4	Rahul	9012345678	rahul@email.com	Nashik
5	Kumari	8899001122	kumari@email.com	Mumbai

COMMODITY

ID	Commodity	Category	Unit	Min °C	Max °C
1	Himachal Apples	Fruit	Kg	0	4
2	Alphonso Mangoes	Fruit	Kg	10	13
3	Nashik Grapes	Fruit	Kg	0	2
4	Frozen Green Peas	Frozen	Kg	-18	-15
5	Pomegranates	Fruit	Kg	5	7

CHAMBER

ID	Name	Type	Capacity	Min °C	Max °C	Status
1	CH01	Chilled	5000	0	5	Active
2	CH02	Frozen	3000	-20	-15	Active
3	CH03	Chilled	4000	0	5	Active
4	CH04	Ambient-Controlled	6000	8	15	Active

LOCATION

ID	Chamber	Code	Capacity
1	1	RACK-A01	1000
2	1	RACK-A02	1000
3	1	RACK-B01	1500
4	2	RACK-F01	800
5	2	RACK-F02	800
6	3	RACK-C01	1200
7	3	RACK-C02	1200
8	4	RACK-D01	1500
9	4	RACK-D02	1500

LOT

Lot	Customer	Commodity	Qty	Arrival	Status
L001	1	1	500	2026-01-10	Released

Lot	Customer	Commodity	Qty	Arrival	Status
L002	2	2	300	2026-01-12	Released
L003	3	3	800	2026-01-15	Stored
L004	1	5	200	2026-02-01	Stored
L005	4	4	450	2026-02-05	Released
L006	5	1	600	2026-02-10	Stored
L007	2	3	350	2026-02-12	Stored
L008	3	2	250	2026-02-15	Stored

ALLOCATION

ID	Lot	Chamber	Location	Start	End	Status
1	L001	1	1	2026-01-10	2026-02-20	Completed
2	L002	4	8	2026-01-12	2026-03-01	Completed
3	L003	1	2	2026-01-15	NULL	Active
4	L004	1	3	2026-02-01	NULL	Active
5	L005	2	4	2026-02-05	2026-03-10	Completed
6	L006	3	6	2026-02-10	NULL	Active
7	L007	3	7	2026-02-12	NULL	Active
8	L008	4	9	2026-02-15	NULL	Active

TEMPERATURE LOG

ID	Allocation	Date/Time	Temp °C	Remarks
1	1	2026-01-11 08:00	2.50	Normal
2	1	2026-01-12 08:00	3.00	Normal
3	1	2026-01-13 08:00	2.80	Normal
4	1	2026-01-20 08:00	5.50	Slightly high
5	2	2026-01-13 09:00	11.50	Normal
6	3	2026-01-16 08:30	1.20	Normal
7	4	2026-02-02 08:00	6.00	Normal
8	5	2026-02-06 10:00	-16.50	Normal
9	5	2026-02-07 10:00	-14.00	Above limit
10	6	2026-02-11 08:00	2.00	Normal

INSPECTION

ID	Allocation	Date	Inspector	Condition	Remarks
1	1	2026-01-15	Mohith	Good	No damage observed
2	1	2026-02-05	Suresh Patil	Good	Ready for release
3	2	2026-01-20	Anita Deshmukh	Good	Quality maintained
4	3	2026-01-25	Rajesh Kumar	Average	Minor moisture
5	5	2026-02-10	Suresh Patil	Good	Frozen state intact

RELEASE

ID	Lot	Date	Qty	Remarks
1	L001	2026-02-20	500	Full release as requested
2	L002	2026-03-01	300	Full release
3	L005	2026-03-10	450	Full release after inspection

INVOICE

ID	Release	Date	Amount	Status
1	1	2026-02-20	10250.00	Paid
2	2	2026-03-01	4800.00	Paid
3	3	2026-03-10	8640.00	Unpaid

PAYMENT

ID	Invoice	Date	Amount	Mode	Status
1	1	2026-02-21	10250.00	Bank Transfer	Completed
2	2	2026-03-02	4800.00	UPI	Completed
3	3	2026-03-12	4000.00	Cash	Completed
4	3	2026-03-15	4640.00	Bank Transfer	Completed

RATE

ID	Commodity	Chamber Type	Rate/Unit/Day	From	To
1	1	Chilled	0.50	2025-01-01	NULL
2	2	Ambient-Controlled	0.40	2025-01-01	NULL
3	3	Chilled	0.55	2025-01-01	NULL
4	4	Frozen	0.80	2025-01-01	NULL
5	5	Chilled	0.45	2025-01-01	NULL

10. DATABASE CONSTRAINTS AND INTEGRITY

Constraint	Use in project
PRIMARY KEY	Uniquely identifies each record in every implemented table.
FOREIGN KEY	Maintains links such as Lot → Customer, Allocation → Lot/Chamber/Location, Payment → Invoice.
NOT NULL	Prevents missing mandatory values such as customer name, commodity name, quantity, dates, and identifiers.
UNIQUE	Prevents duplicate commodity names and chamber names; also ensures one invoice per release in the supplied design.
CHECK	Validates ranges such as minimum temperature < maximum temperature, positive capacity, positive quantity, and positive rates.
DEFAULT	Provides default operational values such as Active/Stored/Unpaid/Completed where specified.

11. NORMALIZATION

The schema separates independent subjects into individual relations instead of storing all information in one large table. Customer details are stored in Customer, commodity characteristics in Commodity, storage facilities in Chamber/Location, and transactional information in Allocation, Release, Invoice, and Payment. Foreign keys connect the relations without repeatedly storing descriptive information.

Normal Form	Project-level interpretation
1NF	Attributes are represented as atomic values; repeating groups are separated into related tables such as Temperature_Log and Payment.
2NF	Non-key attributes in each relation describe the whole primary key. The design uses single-attribute primary keys for the implemented tables.
3NF	Descriptive attributes are kept with their appropriate entities rather than mixing customer, commodity, chamber, and payment details in transaction tables.

12. SQL IMPLEMENTATION

The supplied SQL creates the ColdStorageDB database, creates the 12 implemented tables, inserts sample records, and executes SELECT * queries to verify the contents. The core DDL is summarized below; the full supplied SQL is included in Appendix A.

Stage	SQL purpose
Database setup	DROP DATABASE IF EXISTS, CREATE DATABASE, USE
Schema creation	CREATE TABLE with primary keys, foreign keys, constraints and defaults
Data population	INSERT INTO statements for master and transaction data
Verification	SELECT * FROM each implemented table

13. IMPLEMENTATION OUTPUTS / SCREENSHOTS

The following screenshots are included as evidence of the populated MySQL Workbench tables supplied with the project.

	Allocation_ID	Lot_ID	Chamber_ID	Location_ID	Start_Date	End_Date	Status
▶	1	L001	1	1	2026-01-10	2026-02-20	Completed
	2	L002	4	8	2026-01-12	2026-03-01	Completed
	3	L003	1	2	2026-01-15	NULL	Active
	4	L004	1	3	2026-02-01	NULL	Active
	5	L005	2	4	2026-02-05	2026-03-10	Completed
	6	L006	3	6	2026-02-10	NULL	Active
	7	L007	3	7	2026-02-12	NULL	Active
	8	L008	4	9	2026-02-15	NULL	Active
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Allocation table

	Chamber_ID	Chamber_Name	Chamber_Type	Capacity	Minimum_Temperature	Maximum_Temperature	Status
▶	1	CH01	Chilled	5000	0.00	5.00	Active
	2	CH02	Frozen	3000	-20.00	-15.00	Active
	3	CH03	Chilled	4000	0.00	5.00	Active
	4	CH04	Ambient-Controlled	6000	8.00	15.00	Active
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Chamber table

	Commodity_ID	Commodity_Name	Category	Unit	Minimum_Storage_Temperature	Maximum_Storage_Temperature
▶	1	Himachal Apples	Fruit	Kg	0.00	4.00
	2	Alphonso Mangoes	Fruit	Kg	10.00	13.00
	3	Nashik Grapes	Fruit	Kg	0.00	2.00
	4	Frozen Green Peas	Frozen	Kg	-18.00	-15.00
	5	Pomegranates	Fruit	Kg	5.00	7.00
*	NULL	NULL	NULL	NULL	NULL	NULL

Commodity table

	Customer_ID	Customer_Name	Phone	Email	Address	City
▶	1	Kohli	9876543210	kohli@email.com	12 Market Road	Nagpur
	2	Pawan	9123456780	pawan@email.com	45 Industrial Area	Pune
	3	Vijay	9988776655	vijay@email.com	78 Wholesale Lane	Hyderabad
	4	Rahul	9012345678	rahul@email.com	23 Farm Road	Nashik
	5	Kumari	8899001122	kumari@email.com	56 Cold Street	Mumbai
	6	sai jishnuu	9951317155	jishnuu@gmail.com	g2 greenfield lane	hyderabad
*	NULL	NULL	NULL	NULL	NULL	NULL

Customer table

	Inspection_ID	Allocation_ID	Inspection_Date	Inspector_Name	Condition	Remarks
▶	1	1	2026-01-15	Mohith	Good	No damage observed
	2	1	2026-02-05	Suresh Patil	Good	Ready for release
	3	2	2026-01-20	Anita Deshmukh	Good	Quality maintained
	4	3	2026-01-25	Rajesh Kumar	Average	Minor moisture
	5	5	2026-02-10	Suresh Patil	Good	Frozen state intact
*	NULL	5	NULL	NULL	NULL	NULL

Inspection table

	Invoice_ID	Release_ID	Invoice_Date	Total_Amount	Invoice_Status
▶	1	1	2026-02-20	10250.00	Paid
	2	2	2026-03-01	4800.00	Paid
	3	3	2026-03-10	8640.00	Unpaid
✱	NULL	NULL	NULL	NULL	NULL

Invoice table

	Lot_ID	Customer_ID	Commodity_ID	Quantity	Arrival_Date	Status
▶	L001	1	1	500.00	2026-01-10	Released
	L002	2	2	300.00	2026-01-12	Released
	L003	3	3	800.00	2026-01-15	Stored
	L004	1	5	200.00	2026-02-01	Stored
	L005	4	4	450.00	2026-02-05	Released
	L006	5	1	600.00	2026-02-10	Stored
	L007	2	3	350.00	2026-02-12	Stored
	L008	3	2	250.00	2026-02-15	Stored
✱	NULL	NULL	NULL	NULL	NULL	NULL

Lot table

	Payment_ID	Invoice_ID	Payment_Date	Amount	Payment_Mode	Transaction_Reference	Payment_Status
▶	1	1	2026-02-21	10250.00	Bank Transfer	TXN987654321	Completed
	2	2	2026-03-02	4800.00	UPI	UPI445566778	Completed
	3	3	2026-03-12	4000.00	Cash	CASH001122	Completed
	4	3	2026-03-15	4640.00	Bank Transfer	TXN112233445	Completed
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Payment table

	Rate_ID	Commodity_ID	Chamber_Type	Rate_Per_Unit_Per_Day	Effective_From	Effective_To
▶	1	1	Chilled	0.50	2025-01-01	NULL
	2	2	Ambient-Controlled	0.40	2025-01-01	NULL
	3	3	Chilled	0.55	2025-01-01	NULL
	4	4	Frozen	0.80	2025-01-01	NULL
	5	5	Chilled	0.45	2025-01-01	NULL
✱	NULL	NULL	NULL	NULL	NULL	NULL

Rate table

	Release_ID	Lot_ID	Release_Date	Quantity_Released	Remarks
▶	1	L001	2026-02-20	500.00	Full release as requested
	2	L002	2026-03-01	300.00	Full release
	3	L005	2026-03-10	450.00	Full release after inspection
✱	NULL	NULL	NULL	NULL	NULL

Release table

	Log_ID	Allocation_ID	Recorded_DateTime	Temperature	Remarks
▶	1	1	2026-01-11 08:00:00	2.50	Normal
	2	1	2026-01-12 08:00:00	3.00	Normal
	3	1	2026-01-13 08:00:00	2.80	Normal
	4	1	2026-01-20 08:00:00	5.50	Slightly high
	5	2	2026-01-13 09:00:00	11.50	Normal
	6	3	2026-01-16 08:30:00	1.20	Normal
	7	4	2026-02-02 08:00:00	6.00	Normal
	8	5	2026-02-06 10:00:00	-16.50	Normal
	9	5	2026-02-07 10:00:00	-14.00	Above limit
	10	6	2026-02-11 08:00:00	2.00	Normal
✱	NULL	NULL	NULL	NULL	NULL

Temperature Log table

	Location_ID	Chamber_ID	Location_Code	Capacity
▶	1	1	RACK-A01	1000
	2	1	RACK-A02	1000
	3	1	RACK-B01	1500
	4	2	RACK-F01	800
	5	2	RACK-F02	800
	6	3	RACK-C01	1200
	7	3	RACK-C02	1200
	8	4	RACK-D01	1500
	9	4	RACK-D02	1500
✱	NULL	NULL	NULL	NULL

Location table

14. TESTING AND VALIDATION

Test ID	Test	Expected result	Observed / supplied evidence
T01	Create database and tables	ColdStorageDB and tables created	SQL script supplied for database and table creation.
T02	Insert customer records	Customer rows stored	Customer screenshot shows populated records.
T03	Insert commodity/chamber/location data	Storage master data stored	Screenshots show populated tables.
T04	Insert lots and allocations	Lots are linked to customers and storage	Lot and Allocation screenshots show related identifiers.
T05	Record temperature logs	Multiple observations can be linked to an allocation	Temperature_Log screenshot shows 10 records.
T06	Record inspections	Inspection history is linked to allocations	Inspection screenshot shows 5 records.
T07	Release and billing	Release generates linked invoice	Release and Invoice screenshots show matching IDs.
T08	Record payments	Payments reference invoices	Payment screenshot shows four payment records.
T09	Constraint validation	Invalid values should be rejected	DDL contains CHECK, FK, NOT NULL and UNIQUE constraints.

15. RESULTS

The implemented database successfully represents a cold-storage operational workflow from customer and commodity registration through lot allocation, environmental monitoring, inspection, release, invoicing, and payment. The sample data demonstrates relationships across the tables, including multiple payments against the same invoice and multiple monitoring records against an allocation.

16. CONCLUSION

The Cold Storage Commodities and Chamber Management System provides a structured relational solution for managing storage operations. The ER model and MySQL schema separate major business entities and connect them through keys, while integrity constraints improve reliability. The sample data and MySQL Workbench outputs demonstrate that the database can store and retrieve operational information across the complete storage-to-payment flow.

17. FUTURE ENHANCEMENTS

Enhancement	Benefit
Implement the missing Receipt table if required by the course specification	Align the physical database with the stated 13-table requirement.
Role-based login for administrators and staff	Control access to sensitive operations.
Automatic temperature alerts	Notify staff when recorded temperature moves outside the permitted range.
Capacity validation during allocation	Prevent allocation when a location/chamber has insufficient capacity.
Automatic invoice calculation from rate × quantity × duration	Reduce manual billing errors.
Dashboard and reports	Provide quick views of active lots, chamber occupancy, temperature exceptions, unpaid invoices, and releases.
IoT sensor integration	Capture temperature readings automatically.
Audit logging	Track who changed important operational records.

18. REFERENCES

Reference	Use
MySQL documentation	SQL syntax, DDL, constraints, keys, and relational database implementation.
Course DBMS material	ER modelling, relational design, normalization, and database concepts.
Project implementation	Supplied ER diagram, SQL script, sample data, and MySQL Workbench screenshots.

APPENDIX A — SQL IMPLEMENTATION

The following is the supplied schema/verification code. The INSERT statements are represented by the project data tables in Section 9 to keep this report readable.

```
DROP DATABASE IF EXISTS ColdStorageDB;
CREATE DATABASE ColdStorageDB;
USE ColdStorageDB;

CREATE TABLE Customer (
  Customer_ID INT PRIMARY KEY,
  Customer_Name VARCHAR(100) NOT NULL,
  Phone VARCHAR(15) NOT NULL,
  Email VARCHAR(100),
  Address VARCHAR(200),
  City VARCHAR(50) NOT NULL
);

CREATE TABLE Commodity (
  Commodity_ID INT PRIMARY KEY,
  Commodity_Name VARCHAR(50) NOT NULL UNIQUE,
  Category VARCHAR(50) NOT NULL,
  Unit VARCHAR(20) NOT NULL,
  Minimum_Storage_Temperature DECIMAL(5,2) NOT NULL,
  Maximum_Storage_Temperature DECIMAL(5,2) NOT NULL,
  CHECK (Minimum_Storage_Temperature < Maximum_Storage_Temperature)
);

CREATE TABLE Chamber (
  Chamber_ID INT PRIMARY KEY,
  Chamber_Name VARCHAR(50) NOT NULL UNIQUE,
  Chamber_Type VARCHAR(50) NOT NULL,
  Capacity INT NOT NULL,
  Minimum_Temperature DECIMAL(5,2) NOT NULL,
  Maximum_Temperature DECIMAL(5,2) NOT NULL,
  Status VARCHAR(20) NOT NULL DEFAULT 'Active',
  CHECK (Minimum_Temperature < Maximum_Temperature),
  CHECK (Capacity > 0)
);

CREATE TABLE Rate (
  Rate_ID INT PRIMARY KEY,
  Commodity_ID INT NOT NULL,
  Chamber_Type VARCHAR(50) NOT NULL,
  Rate_Per_Unit_Per_Day DECIMAL(10,2) NOT NULL,
  Effective_From DATE NOT NULL,
  Effective_To DATE,
  FOREIGN KEY (Commodity_ID) REFERENCES Commodity(Commodity_ID),
  CHECK (Rate_Per_Unit_Per_Day > 0)
);

CREATE TABLE Lot (
  Lot_ID VARCHAR(10) PRIMARY KEY,
  Customer_ID INT NOT NULL,
  Commodity_ID INT NOT NULL,
  Quantity DECIMAL(10,2) NOT NULL,
  Arrival_Date DATE NOT NULL,
  Status VARCHAR(20) NOT NULL DEFAULT 'Stored',
  FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
  FOREIGN KEY (Commodity_ID) REFERENCES Commodity(Commodity_ID),
  CHECK (Quantity > 0)
);

CREATE TABLE Location (
  Location_ID INT PRIMARY KEY,
  Chamber_ID INT NOT NULL,
  Location_Code VARCHAR(20) NOT NULL,
  Capacity INT NOT NULL,
  FOREIGN KEY (Chamber_ID) REFERENCES Chamber(Chamber_ID),
  UNIQUE (Chamber_ID, Location_Code),
  CHECK (Capacity > 0)
);

CREATE TABLE Allocation (
  Allocation_ID INT PRIMARY KEY,
  Lot_ID VARCHAR(10) NOT NULL,
  Chamber_ID INT NOT NULL,
  Location_ID INT NOT NULL,
  Start_Date DATE NOT NULL,
  End_Date DATE,
  Status VARCHAR(20) NOT NULL DEFAULT 'Active',
  FOREIGN KEY (Lot_ID) REFERENCES Lot(Lot_ID),
  FOREIGN KEY (Chamber_ID) REFERENCES Chamber(Chamber_ID),
  FOREIGN KEY (Location_ID) REFERENCES Location(Location_ID)
);

CREATE TABLE Temperature_Log (
  Log_ID INT PRIMARY KEY,
  Allocation_ID INT NOT NULL,
  Recorded_DateTime DATETIME NOT NULL,
  Temperature DECIMAL(5,2) NOT NULL,
```

```

Remarks VARCHAR(200),
FOREIGN KEY (Allocation_ID) REFERENCES Allocation(Allocation_ID)
);

CREATE TABLE Inspection (
Inspection_ID INT PRIMARY KEY,
Allocation_ID INT NOT NULL,
Inspection_Date DATE NOT NULL,
Inspector_Name VARCHAR(100) NOT NULL,
`Condition` VARCHAR(50) NOT NULL,
Remarks VARCHAR(200),
FOREIGN KEY (Allocation_ID) REFERENCES Allocation(Allocation_ID)
);

CREATE TABLE `Release` (
Release_ID INT PRIMARY KEY,
Lot_ID VARCHAR(10) NOT NULL,
Release_Date DATE NOT NULL,
Quantity_Released DECIMAL(10,2) NOT NULL,
Remarks VARCHAR(200),
FOREIGN KEY (Lot_ID) REFERENCES Lot(Lot_ID),
CHECK (Quantity_Released > 0)
);

CREATE TABLE Invoice (
Invoice_ID INT PRIMARY KEY,
Release_ID INT NOT NULL UNIQUE,
Invoice_Date DATE NOT NULL,
Total_Amount DECIMAL(12,2) NOT NULL,
Invoice_Status VARCHAR(20) NOT NULL DEFAULT 'Unpaid',
FOREIGN KEY (Release_ID) REFERENCES `Release`(Release_ID),
CHECK (Total_Amount >= 0)
);

CREATE TABLE Payment (
Payment_ID INT PRIMARY KEY,
Invoice_ID INT NOT NULL,
Payment_Date DATE NOT NULL,
Amount DECIMAL(12,2) NOT NULL,
Payment_Mode VARCHAR(30) NOT NULL,
Transaction_Reference VARCHAR(50),
Payment_Status VARCHAR(20) NOT NULL DEFAULT 'Completed',
FOREIGN KEY (Invoice_ID) REFERENCES Invoice(Invoice_ID),
CHECK (Amount > 0)
);

-- INSERT statements supplied in the project populate all 12 implemented tables.
-- Verification:
SELECT * FROM allocation;
SELECT * FROM chamber;
SELECT * FROM commodity;
SELECT * FROM customer;
SELECT * FROM inspection;
SELECT * FROM invoice;
SELECT * FROM location;
SELECT * FROM lot;
SELECT * FROM payment;
SELECT * FROM rate;
SELECT * FROM `release`;
SELECT * FROM temperature_log;

```

APPENDIX B — SUBMISSION CHECKLIST

Item	Status
Project title included	✓
Student details included	✓
Abstract and introduction	✓
Problem statement and objectives	✓
Scope and requirements	✓
Technology stack	✓
ER diagram	✓
Relational schema / table definitions	✓
Sample data in tabular form	✓
Constraints and normalization	✓
SQL implementation summary and appendix	✓
MySQL Workbench screenshots	✓
Testing and results	✓
Conclusion and future scope	✓
Receipt table mismatch explicitly identified	■ Needs correction if rubric requires Receipt as an implemented table

Final quality-control note: before submission, verify whether your lecturer's official report template requires additional front matter (certificate, acknowledgement, table of contents, list of figures/tables, or signature pages). If so, those pages should be inserted according to the official template. The technical content in this report is grounded in the SQL, ER diagram, and screenshots supplied for this project.